Advanced Cryptography Elisabeth Oswald

Secret Sharing

2025

1 Introduction

We mentioned before in this course that MPC can be realised via either garbled circuits or secret sharing. Both approaches have different advantages and disadvantages in terms of computation and communication cost.

Secret sharing is one of the fundamental techniques that underpins practical approaches for MPC with more than two parties. We now look at how secret sharing works.

2 Defining Secret Sharing

Intuitively speaking we are interested in a cryptographic mechanism that enables us to "split up" some secret value into n shares, in such a way that no single share, and no subset of $t < n$ shares reveals any information about the secret value. We must always be able to reconstruct the secret correctly when using all n shares, and we may be interested in schemes where t shares are sufficient for reconstruction.

Firstly let us capture the functions that we are looking for more formally.

Given a message space $\mathcal{M}$, a threshold value $t \in \mathbb{N}$, and a number of shares $n \in \mathbb{N}$, with $t < n$.
A (t, n) **threshold secret sharing scheme** consists of two algorithms:

`Share:` takes a message $m \in \mathcal{M}$ as input, and outputs an n-tuple of randomly selected shares $\mathbf{s} = (s_1, \ldots, s_n)$.

`Reconstruct:` takes a subset of t or more shares as input, and produces a message m' as output.

Notice that in our functional specification we simply say that `Reconstruct` returns an element from the space of messages that our scheme is defined over. Obviously, we will be interested in schemes that return the original message upon being given n correct shares, and potentially t shares from an "authorised subset" of shares.

We will not spend much time on considering access structures in this course. But it is probably helpful to reflect a bit on the practical situation where shares are distributed to different entities with the aim of enabling a subset of entities to reconstruct the secret. Perhaps it is even the case that some entities should have "more power" than other entities. Wikipedia [1] gives the example of a fictive company structure, where the president should be able to reveal the secret or any three board members. Suppose that the company has a board consisting of 5 members. Then we would like to have a $(3, 8)$ secret sharing scheme: each board member gets one share (thus we need 5 shares for the board) and the president gets 3 shares. Thus in total we need $5 + 3 = 8$ shares, and we require a scheme where any set that consists of either at least three board members, or includes the president, enables reconstruction of the secret.

Let us try and formalise the situation from above. We call $\mathcal{U}$ a subset of the users of a scheme. Each user has at least one share, so the set of shares that belong to $\mathcal{U}$ is the set $\{s_i, i \in \mathcal{U}\}$. If the set of users $\mathcal{U}$ has sufficiently many shares $|\mathcal{U}| > t$, then we say that the set has authorisation. Now the correctness goal of (t, n) threshold secret sharing scheme is to allow any set $\mathcal{U}$ that has authorisation to reconstruct the secret.

> A (t, n) threshold secret sharing scheme is **correct** if, for all authorised sets of user $\mathcal{U}$, s.t. $|\mathcal{U}| > t$), and for all $\mathbf{s} \leftarrow \texttt{Share}(m)$, we have that $\texttt{Reconstruct}(\{s_i, i \in \mathcal{U}\}) = m$.

Defining security. We would like that a secret sharing scheme does not reveal any information about the value that is being secret shared. Thus informally speaking we would like to formalise a property along the lines of:
" an adversary cannot distinguish between shares of a message and random stuff", or
" an adversary cannot distinguish between the shares of two different messages". (Notice that this is basically the same property that we ask for when considering secrecy of encryptions.)

It is common to formalise the "second variation" as follows.

> Let Σ be a (t, n) threshold secret sharing scheme with message space $\mathcal{M}$. We say that Σ is **perfectly secure** if for all choices of messages $m, m' \in \mathcal{M}$, no unauthorised subset of users $\mathcal{U}$ can distinguish between the shares of m and m'.

To really understand this definition in detail, it can help to try and write down this as a game (either with a visual aid, or even via an algorithm), where we are forced to make

[1] `https://en.wikipedia.org/wiki/Secret_sharing`

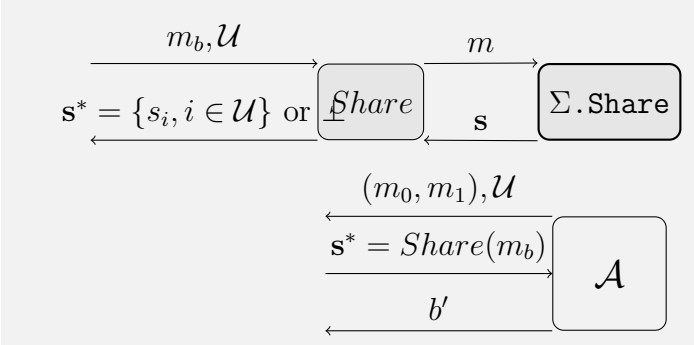

Figure 1: Exp ind-shares

things rather precise. Let's start off with a visual aid, which I provide in Figure 1. We can see that the game has three algorithms: the algorithm Σ.`Share` (from the scheme), an interface to this algorithm called *Share*, and the adversary $\mathcal{A}$. In the definition, we don't allow the adversary to interact with the Σ.`Share` oracle (algorithm). Therefore, we have no interaction indicated in the picture. The working principle of the Σ.`Share` function is also public knowledge. But any randomness that may be sampled as part of the function is not know to the adversary. You can think of the situation like this: the adversary has the implementation of the function, but they cannot know the execution values for a specific input, because every time the function executes, it randomly chooses some values, and the rest of the run depend on those values.

The *Share* function, takes one of the two messages as input (they choice is made randomly here), and a set of users. In the game we only care about a set of unauthorised users (a set of authorised users can of course learn information about the secret message). The *Share* function thus checks the set of users generated by the adversary and it returns $\perp$ (a symbol for "no output") if the set of users is actually authorised. Otherwise it will return the shares for the set $\mathcal{U}$, by a call to the share function of the scheme Σ.`Share`.

The game itself consists just of the messages that the adversary produces:

1. they may select the two messages m_0, m_1 and an unauthorised set of users,
2. then they are given the shares of one of the messages (assuming they produced a valid set)
3. and then they must decide which message the shares correspond to.

Whenever this game is started, the adversary chooses two messages, thus if the game is played more than once, they may elect to choose the same messages. However, in the Σ.`Share` function, a random set of shares is produced: i.e. even if the adversary supplies the same message more than once, they will always see a new random set of shares.

2.1 Sharing is not splitting

It sometimes helps to look at an example that is obviously insecure. Consider then a scheme where the secret is split into different, equal length chunks, and each user is randomly assigned a chunk. There is actually no way to reconstruct the secret here unless we save the random choice that we make in the assignment of shares to users, and give this to the reconstruct algorithm. This may not particularly helpful in a practical setting, but we ignore this as the approach itself is fundamentally flawed anyway. We can express the $\Sigma.\texttt{Share}(m)$ functionality by the following algorithm.

```
1  m = m_1||...||m_n
2  for i = 1 to n do
3     s_i <-$ m_j
4  return (s_1||...||s_n);
```

Listing 1: An insecure $\Sigma.\texttt{Share}(m)$

Before trying to semi-formally argue why this idea is bad, we consider it from a "down to earth" point of view. Obviously if we simply "given different parts of the secret" to different users then each user learns a part of the secret. Thus, if an adversary takes over a user, they learn something too. The approach is completely flawed.

How will this show when we use the "semi-formal" game from before? The $\Sigma.\texttt{Share}(m)$ function in the game will now return a "random" assignment from parts of the input message m_b to users, and the *Share* function will make a small enough subset available to the adversary. It is therefore easy for the adversary to choose two messages that are easily distinguished by just getting some parts. The adversary can simply choose one message to be a string consisting of all 1 characters, and the other message to be a string consisting of all 0 characters. Therefore, if the "all 1" message gets secret shared, all parts consist again of only 1 characters, and if the "all 0" message gets secret shared, all parts consist of only 0 characters. The adversary will thus easily recognise (even when making just a single call to the *Share* function) which of the two messages was shared out.

3 Boolean secret sharing: an (n, n) scheme

Next we look at the (perhaps) simplest scheme that there is: we initially restrict ourselves to a scheme with only 2 shares.

We assume that the purpose of the scheme is to work on bit strings. Therefore, we will use the bit-wise addition modulo 2 (aka Boolean addition, aka exclusive-or) to create two shares for an input message. The share function randomly selects a value for the first share (of length equal to the message, say that messages have l bits), and then derives the

second share via an exclusive-or between the first share and the message. The reconstruct function just reverse the Boolean addition using both shares.

```
def B22.Share(m):
    s_1 ←$ {0,1}^l
    s_2 = s_1 ⊕ m
    return (s_1, s_2)
```

Listing 2: Boolean (2,2) secret sharing: Share function

```
def B22.Reconstruct(s_1, s_2):
    return (s_1 ⊕ s_2)
```

Listing 3: Boolean (2,2) secret sharing: Reconstruct function

In this example the sets of unauthorised users are $\{\{1\},\{2\}\}$ (i.e. no single user should learn anything about s). The only authorised set is thus $\{1,2\}$ (it includes both users).

Correctness follows immediately from the construction: the reconstruct function computes $s_1 \oplus s_2 = s_1 \oplus (s_1 \oplus m) = m$.

3.1 Security proofs

Now let us reason about the security of this construction, first by taking a "down to earth" (= super informal) perspective. The share s_1 is just a random string that is independently chosen of the message. Clearly it cannot contain any information about the message. The share s_2 is an exclusive-or between the message and the random string s_1. Given that we are randomly choosing s_1 every time we call share, we essentially have a one-time encryption of m. We already know that this is exactly the scheme for which we can prove perfect secrecy even against unbounded adversaries. Therefore we know that also s_2 on its own does not reveal any information about m, which gives us the desired property of perfect secrecy for the scheme.

Let us make this more formal. We know consider a single run of this protocol, and thus the choice of an adversary $(m, m'), \mathcal{U}$. Let us assume that the adversary indeed supplies a set of unauthorised users, which must be a set of a single user. We start with the case where the adversary sees share s_1. The probability for share one to take value a when the message m was shared is the same as when message m' was shared because s_1 is always chosen uniformly at random. Thus the adversary cannot distinguish whether m or m' was shared. Now we consider the case where the adversary sees the share s_2. The share $s_2 = s_1 \oplus m$ and thus $s_1 = m \oplus s_2 = a$, and also $s_2' = s_1' \oplus m'$ and therefore $s_1' = m + oplus s_2' = a$. Because all a are equally likely we have that

$$\Pr[s_2 = a|m] = 1/2^l = \Pr[s_2' = a|m'].$$

We can also cast a somewhat more formal argument by looking at how the returns from a call to a left oracle are indistinguishable from the calls to a right oracle. For this we first need to write the code for the oracle:

```
1  Share(m_L, m_R):
2      if U ≥ 2 return perp
3      s_1 ←$ {0,1}^l
4      s_2 = s_1 ⊕ m_L
5      return {s_i | i ∈ U}
```

We now need to show, in a sequence of steps, that a call to the left oracle can be replaced by a call to the right oracle, i.e. that we can switch out m_L in line four to m_R.

Firstly, notice that we can expand out the if statement without changing the behaviour of the left oracle, and then notice that for the first branch, which equals the case where the adversary gets s_1, we can replace m_L with m_R.

```
1  Share(m_L, m_R):
2      if U ≥ 2 return perp
3      if U = {1}
4          s_1 ←$ {0,1}^l
5          s_2 = s_1 ⊕ m_R
6          return s_1
7      if U = {2}
8          s_1 ←$ {0,1}^l
9          s_2 = s_1 ⊕ m_L
10         return s_2
```

Secondly, notice that in the second branch, the code really corresponds to one-time pad encryption of m_L. Recall that the result of a one-time pad encryption is not distinguishable from a random string, so we can in fact replace $s_2 = s_1 \oplus m_L$ with $s_2 = s_1 \oplus m_R$. This is the same as saying that the OTP satisfies the notion of LR security (we showed this before).

```
1  Share(m_L, m_R):
2      if U ≥ 2 return perp
3      if U = {1}
4          s_1 ←$ {0,1}^l
5          s_2 = s_1 ⊕ m_R
6          return s_1
7      if U = {2}
8          s_1 ←$ {0,1}^l
9          s_2 = s_1 ⊕ m_R
10         return s_2
```

Now we really have the code for a call to the right oracle. Hence we have shown that a call to the left oracle can be replaced without changing anything about its outputs with a call to the right oracle. Which concludes our argument.

3.1.1 Generalisations

It is easy to see that we can generalise this construction from using the "addition modulo 2" to the "addition modulo p". The correctness and security properties remain the same, which is useful, because many real world protocols, where secret sharing might be useful, work with numbers modulo p. It is also easy to extend this to an (n, n) context.

It is possible, albeit tedious, to consider the generalisation to the $(2, n)$ case. To achieve this, we basically construct a (2,2) secret sharing scheme for each pair of users. Evidently, we need many shares. Going then towards a (t, n) scheme could be done in a similar vein, by constructing a (t, t) scheme for any authorised set of users, leading to an inefficient construction again. Clearly, a better idea is now required.

4 Shamir (t,n) threshold secret sharing

Such a better idea occurred to the "crypto Maverick" Adi Shamir. He realised a connection with polynomials—in particular the fact $d + 1$ points of a (secret) polynomial enable to reconstruct that secret polynomial uniquely—could be a useful tool in realising a (t, n) secret sharing scheme. To understand his idea and the resulting construction, we first make a short detour into polynomials.

4.1 Polynomial (interpolation)

A **polynomial** is a mathematical expression where (finitely many) variables and coefficients are added, multiplied, subtracted and raised to a non-negative power. For instance a polynomial in the variable x is written as

$$f(x) = a_d x^d + a_{d-1} x^{d-1} + \ldots a_1 x + a_0.$$

The highest non-zero term determines the **degree** of a polynomial, e.g. $f(x) = x^2 - 1$ has degree 2.

We can evaluate a polynomial by "plugging in" a concrete value for x over some concrete algebraic structure (e.g. the real numbers, complex numbers, or for us the most important option: a finite group/ring/field). For instance, we can evaluate $f(x) = x^2 - 1$ over the

real numbers by selecting $x = 2 : f(2) = 3$; we can also evaluate it over $\mathbb{Z}_3$ in which case $f(2) \equiv 0$.

A **root** of a polynomial is a value r for which the polynomial evaluates to zero: $f(r) = 0$. In the previous example, $r = 2$ was a root of $x^2 - 1$ over $\mathbb{Z}_3$.

Note that when working modulo p, because $x^p \equiv x \,(\text{mod } p)$, we always assume that exponents are strictly smaller than p (otherwise we reduce the exponent modulo $p - 1$); with this we ensure that there is a unique function expressing a polynomial. When working with polynomials modulo p we want to ensure that p is prime (so coefficients have inverses). When working over say $\mathbb{Z}_p$ it holds that the algebraic structure that is defined as the set of polynomials over $\mathbb{Z}_p$ is a commutative ring: i.e. if we take two polynomials over $\mathbb{Z}_p$ then their sum exist, their difference exist, and we can multiply two polynomials, and there is a division with remainder. For instance dividing $6x^4 + 8x + 1$ by $2x^2 + 4$ over $\mathbb{Z}_1 1$ gives $(3x^2 + 5)(2x^2 + 4) + (8x + 3)$.

There are two useful properties of polynomials (applicable both to polynomials over the reals, but also over finite groups/rings/fields).

Property 1. A non-zero polynomial of degree d has at most d roots.

Property 2. Given $d+1$ distinct pairs $(x_1, y_1), \ldots, (x_{d+1}, y_{d+1})$, there is a unique polynomial $f(x)$ such that $f(x_i) = y_i$ for $1 \leq i \leq d + 1$.

We need one further property in the context of a cryptographic application for polynomials, which is the number of polynomials of a given degree in dependence over how many points we know of the polynomial. Size matters in cryptography: we need problems that we use to hide secrets to be computationally hard, which is a way of saying that we need to construct problems in such a way that brute force searching through solutions will not succeed in practice.

So let us consider the number of polynomials over a given finite structure with p elements. A degree d polynomial has $d + 1$ coefficients, thus, without any information there are p choices for each coefficient and thus there are p^{d+1} polynomials. Suppose we have d points given: these will enable us to fix all but one coefficient, implying that there are p polynomials. Similarly, if we have $d - 1$ pairs given they determine all but two coefficients, meaning that there are p^2 polynomials left. Therefore, if $d-k$ pairs of points are given, then there are p^{d+1-k} polynomials that go through these points. To have a

computationally hard problem even for a small number of points, we must choose p to be a very large number (in the area of an RSA composite modulus).

4.1.1 Lagrange interpolation

Now we look at how to (efficiently) find a polynomial $f(x)$ when given pairs of points $(x_i, f(x_i)$. Let us start with a toy example so that we get an idea of how to solve this problem. Suppose we have the pairs $(1, f(1) = 2), (2, f(2) = 0), (6, f(6) = 7), (8, f(8) = 10)$ where the coefficients are from $\mathbb{Z}_{11}$. Because we have four pairs, we know that there is a unique polynomial of degree 3 $f(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ over $\mathbb{Z}_{11}$ with these four points. Plugging our four pair into $f(x)$ we get a set of four equations;

$$\begin{aligned} 2 &= a_3 + a_2 + a_1 + a_0 \\ 0 &= 8a_3 + 4a_2 + 2a_1 + a_0 \\ 7 &\equiv 7a_3 + 3a_2 + 6a_1 + a_0 \\ 10 &\equiv 6a_3 + 8a_1 + a_0 \end{aligned}$$

This is a set of linear equations with a unique solution (we have as many equations as we have unknowns), and any technique (Gaussian elimination, matrix inversion) can be used to get the values for the a_is. In particular we can express $a_0 = 2 - a_3 - a_2 - a_1$ using the first equation and $a_1 = -2 - 7a_3 - 3a_2$ using the second equation. Plugging both of these into the third equation turns the third equation into $4 = 4a_3 + 9a_2$ and it turns the fourth equation into $22 \equiv -13a_2$, which implies that $a_2 \equiv 0 \,(\text{mod } 11)$. Now we can use this to infer that $a_3 \equiv 1$, $a_1 \equiv 2$ and $a_0 \equiv 10$. Thus the unique degree three polynomial is given as $f(x) = x^3 + 2x + 10 \,(\text{mod } 11)$.

In this example, doing a "manual" elimination is easy because we work with a small modulus. This implies that all arithmetic operations, including inversions, are fast. But, as we discussed before, this would not be the case in practice where we must choose a very large modulus. Therefore it is useful to have more ways of solving the same problem, and the so-called Lagrange interpolation is a well liked technique.

A technique that is efficient even with large moduli goes back to Lagrange. It is based on first constructing the so-called Lagrange polynomials

$$\delta_i(x) = \frac{\prod_{j \neq i}(x - x_j)}{\prod_{j \neq i}(x_i - x_j)}.$$

These polynomials have the property that given a pair $x_i, f(x_i)$, the polynomial $\delta_i(x_1)$ evaluates to 1 and it evaluates to 0 otherwise. This is because, calling the numerator $q(x)$, the denominator is $q(x_i)$. Thus if we evaluate $\delta_i(x)$ on x_i it will evaluate to 1.

Example. Consider the pairs of points from the previous example. Then for the first point $(1, 2)$, the corresponding Lagrange polynomial is

$$\delta_1(x) = \frac{(x-2)(x-6)(x-8)}{(1-2)(1-6)(1-8)} \equiv (x^3 + 6x^2 + 10x + 3)9^{-1}.$$

The idea for polynomial interpolation using such Lagrange polynomials is now as follows: we construct a Lagrange polynomial for each given point on the polynomial. Each of the Lagrange polynomials satisfies that they evaluate to $x_i, 1$ per construction and to $x_j, 0$ for all $x_j \neq x_i$. We want each of them however to evaluate to $x_i, f(x_i)$ which is easy to achieve by multiplying with $f(x_i)$. Now the sum of the Lagrange polynomials scaled individually by $f(x_i)$ still satisfies the property that each $\delta_i(x)$ evaluates to 0 if $x = x_j$. So the polynomial $f(x)$ that we are looking for is given as:

$$f(x) = \sum_{i=1}^{d+1} f(x_i)\delta_i(x).$$

Note that $f(x_i)$ is NOT a function here but a value in $\mathbb{Z}_p$.

Example. Let us use this technique to reconstruct the polynomial given the four points $(1, f(1) = 2), (2, f(2) = 0), (6, f(6) = 7), (8, f(8) = 10)$. We already derived $\delta_1(x)$ above. Plugging in the remaining points give us:

$$\begin{aligned} \delta_2(x) &= 5(x^3 + 7x^2 + 7x + 7) \\ \delta_3(x) &= 3(x^3 + 4x + 6) \\ \delta_4(x) &= 8(x^3 + 2x^2 + 9x - 1) \end{aligned}$$

The sum over the Lagrange polynomials $f(x_1)\delta_1(x)+f(x_2)\delta_2(x)+f(x_3)\delta_3(x)+f(x_4)\delta_4(x) = -(x^3 + 6x^2 + 10x + 3) + 0 - (x^3 + 4x + 6) + 3(x^3 + 2x^2 + 9x - 1)$ then gives us the final result as $x^3 + 2x + 10$.

As a final note: one can prove that this method always produce a unique solution, which is in accordance to the second property for polynomials that we listed before.

4.2 Shamir's scheme

Now we have all the ingredients to describe Shamir's secret sharing scheme. Suppose that a secret $m \in \mathbb{Z}_p$ is to be shared with a threshold of t. We choose a secret degree $(t-1)$ polynomial f, that satisfies $f(0) \equiv m \pmod p$. This is another way of saying that the coefficient a_0 in our notation is exactly $m \pmod p$. All other coefficients are chosen randomly from $\mathbb{Z}_p$. The $i-$th user receives the point $(i, f(i) \pmod p)$[2]. Because

[2]It is of course possible to give a user more than one share/point if so desired in a concrete situation

of property two we know that there is exactly one polynomial of degree t for a scheme with $t-1$ shares. We have also seen efficient ways of constructing such a polynomial.

```
def Stn.Share(m):
    a_1, ..., a_{t-1} <-$- Z_p
    f(x) = m + sum_{j=1}^{t-1} a_j x^j
    for i=1 to n:
        s_i = (i, f(i) (mod p))
    return s = (s_1, ..., s_n)
```

Listing 4: Shamir (t, n) secret sharing: Share function

```
def Stn.Reconstruct({s_i | i ∈ U}):
    If |U| ≥ t:
        Derive unique degree t-1 polynomial f(x)
        return f(0)
    else
        return ⊥
```

Listing 5: Shamir (t, n) secret sharing: Reconstruct function

Public recombination vector. The reconstruction of the entire secret polynomial is actually not necessary to derive the shared secret. Recall that the secret polynomial is reconstructed as

$$f(x) = \sum_{i=1}^{d+1} f(x_i)\delta_i(x)$$

and $s = f(0)$, therefore the computation of interest is actually

$$f(0) = \sum_{i=1}^{d+1} f(x_i)\delta_i(0).$$

Notice that the $x-$coordinates x_i are public values, and the $f(x_i)$ are the shares held by the players. Therefore, the $\delta_i(0)$ only depend on public values, in particular they only depend on which players come together to reconstruct. The secret s is hence a linear combination of public values and secret shares.

4.2.1 Proving security

In addition to correctness (which we argued informally before) we also wish to show that this construction reveals no further information about the message. We will do this informally at first, then we make it a little bit more formal, and then we consider how this fits with the definition (indistinguishable sharings) that we gave before.

Informal. Observe that m is an element of $\mathbb{Z}_p$, that was chosen with some probability. All the coefficients of $f(x)$ where chosen independently and randomly from $\mathbb{Z}_p$, thus each choice lead to a coefficient with probability $1/p$. Suppose that $t-1$ shares are known to an adversary. We know from the Lagrange reconstruction method that $m = f(0) = \sum_{i=1}^{t} y_i \delta_i(0)$, and therefore $m - y_t\delta_t(0) = \sum_{i=1}^{t-1} y_i\delta_i(0)$. The $\delta_t(0)$ must be non-zero (per its construction: all the x_j are non-zero and also the $x_i - x_j$ are non-zero). Thus the left hand side looks like one-time pad encryption of m still. It in fact is a one-time pad encryption because y_t is from a uniform distribution. In fact all shares are uniformly distributed in $\mathbb{Z}_p$ (per construction of the scheme)[3].This implies that even with $t-1$ shares we have learned nothing in addition to whatever we may have known about the selection of m before.

Note that our reasoning can also be related to the counting argument from before: for any $t-1$ shares there exist a unique degree $t-1$ polynomial, but there are p degree t polynomials that pass through the given $t-1$ points and the missing point. Thus we see again that even if we know $t-1$ shares, we still have p choices for the final share, which are as many choices as we have for the unknown secret. Therefore, we have learnt no additional information.

Towards the security definition. According to our security definition, we want to show that at an adversary cannot distinguish between the shares of two messages m, m'. This can be done by first using a simulation based argument, i.e. where we show that we cannot distinguish between a real execution of the protocol and an execution in an ideal world.

In the case of the secret sharing scheme, an execution in an ideal world would constitute of choosing the shares s_i randomly rather than in relation to the secret polynomial. We can make this idea semi-formal by describing the real and the ideal world via two algorithms, that explain (somewhat) precisely how the worlds work. We then assume that an adversary interacts with the real and ideal world via these algorithms: by this we mean that they can call the algorithm with inputs of their choice, and they observe the output of the algorithm. They also know in principle how the algorithms work, but they don't have access to any internal randomness.

[3]Notice that we choose the coefficients a_i uniformly at random and these define the polynomial, from which we derive the shares. We know that this mapping is bijective: we have a process that given enough shares reconstructs the polynomial uniquely. Therefore, also the shares must have a uniform distribution.

```
1  def Share(m, t, 𝒰 ⊂ {1,...,p}):
2      if |𝒰| < t:
3          return ⊥
4      a_1,...,a_{t-1} ←$ ℤ_p
5      f(x) = m + Σ_{j=1}^{t-1} a_j x^j
6      for i=1 to n:
7          s_i = (i, f(i) (mod p))
8      return {s_i | i ∈ 𝒰}
```

Listing 6: Real world

```
1  def Share(m, t, 𝒰 ⊂ {1,...,p}):
2      if |𝒰| < t:
3          return ⊥
4      for i ∈ 𝒰:
5          y_i ←$ ℤ_p
6          s_i = (i, y_i)
7      return {s_i | i ∈ 𝒰}
```

Listing 7: Ideal world

Now lets consider what happens when a user calls $Share(m, t, \mathcal{U})$ and sees the results $\{(i, y_i) \in \mathcal{U}\}$. To be more precise we are interested in the probability for each output (for each library).

In case that the library that actually corresponds to the Shamir scheme, the probability by which a given input $m, t, \mathcal{U}$ produces and output i, y_i depends on the random choice in line 2, and then the "restriction" that the input puts on the polynomial. Recall that we need the polynomial to run through $(0, m)$ as well as the outputs (i, y_i). We sample $t - 1$ random values from $\mathbb{Z}_p$ in line two, which implies that there are p^{t-1} polynomials possible from that choice in line 2. Then we select one out of them which must go through the given $|\mathcal{U}| + 1$ points; there are exactly $p^{t-1-|\mathcal{U}|}$ such polynomials. Thus the overall probability to select a polynomial with $t - 1$ random coefficients that go through some given $|\mathcal{U}|$ points is

$$\frac{p^{t-1-|\mathcal{U}|}}{p^{t-1}} = p^{-|\mathcal{U}|}.$$

The library corresponding to the ideal world samples $|\mathcal{U}|$ values from $\mathbb{Z}_p$ in line 5. There are $p^{|\mathcal{U}|}$ choices here thus the given set (i, y_i) is sampled with probability $p^{-|\mathcal{U}|}$.

As the inputs $m, t, \mathcal{U}$ where arbitrary, we can conclude that independent of the choice of inputs, both libraries return tuples (i, y_i) with the same probability distribution. Therefore the real and the ideal world are indistinguishable for an adversary. Furthermore,

because the ideal world simply returns tuples with random y_i values, we can conclude that shares are indistinguishable from randomly chosen numbers.

Indistinguishable shares. We can now show that Shamir (t, n) Secret Sharing satisfies our security notion of indistinguishable shares[4]. Consider that an adversary calls the Left Oracle with an input $(m, m', t, \mathcal{U})$. The functionality of the Left Oracle is the same as the Real Oracle, and because Real and Ideal Oracles are indistinguishable, we can "replace" the call to the real world with a call to the Ideal Oracle. In the ideal world library however, the input m is never used. Consequently, this input could be replaced by any other input m'. Now because the ideal and real world libraries are indistinguishable, we can now swap back the call to the real world library, which is now a call with input $(m', t, \mathcal{U})$. We have shown, in a sequence of steps, where no step impacts on the distribution of the library output, that we can swap m and m' inside the Left world. Therefore this is the same as a call to the Right Oracle. Thus, an adversary cannot tell whether the output comes from the Right Oracle or the Left Oracle.

[4]Write a Left and a Right Oracle before reading on.